Proposal presentation – script (4 min)

Matches `mia.pdf` (8 slides). Follows the course template: Problem Statement (1 min) · Questions, Hypotheses and Tests (2 min) · Final Deliverable (1 min). About 500 words, i.e. ∼3:45 at a calm pace (∼140 words/min). Suggested split: **Speaker A** slides 1–3 (∼1:10), **Speaker B** slides 4–7 (∼2:35); move slide 4 to A for a more even split. Slide 8 stays on screen during Q&A.

---

1 · CS-471 Course Project — A (∼10 s)

Hi, we are Mayeul and Sergio, team PolyRack-Beyond Hotness. Our project reproduces the OSDI 2025 paper *Tiered Memory Management Beyond Hotness* and, if time permits, applies its ideas to PolyRack.

2 · Problem Statement (1/2) — A (∼30 s)

Why is it important, what is involved. Modern servers combine two memory tiers: a small, fast DRAM, and a large, cheaper CXL memory that is about two times slower. Whatever does not fit in DRAM goes to the slow tier, and placement and page migration decide what goes there, today based on hotness.

What causes the problem. On the right, each black bar is one cache miss. In a sequential scan the misses are independent, so the out-of-order core overlaps them: four misses cost about one latency. In pointer chasing, each load depends on the previous one, so the misses are serialized: four misses, four latencies. So the real cost of an access is roughly latency divided by memory-level parallelism, MLP.

3 · Problem Statement (2/2) — A (∼30 s)

Take a hot array and a colder linked list. A hotness-based policy puts the array in DRAM and the list in CXL, but then the list's serialized misses pay the full CXL latency. Swapping them is actually faster, because the array's MLP hides most of the slow-tier latency.

Does prior work solve it? No. TPP, NBT, Nomad and Colloid all rank by hotness, and on the paper's benchmark they are 12 to 14% slower than not migrating at all; on a graph workload, Nomad is up to 217% slower.

(hand over to B)

4 · Questions, Hypotheses and Tests (1/3) — B (∼35 s)

These are our questions, in priority order: each one only makes sense if the previous one holds. Q1, the root: is hotness a faithful metric for performance? If not, Q2: can we define a better one? Q3 tests the placement policy built on it, SOAR, and Q4 extends it to PolyRack.

The key insight: the slowdown caused by the slow tier is *how often* we go there, times *how much* each access stalls the core. Hotness only counts the first factor. AOL, amortized offcore latency, is the second one: latency divided by MLP, measured online from hardware counters. With it, the paper predicts slowdown much more accurately.

5 · Questions, Hypotheses and Tests (2/3) — B (∼40 s)

For each question we have a hypothesis, an insight, a test, and the result that would refute it.

Q1: we expect that, with the same DRAM, putting the hot data in DRAM is *slower* than putting the cold data there, because MLP hides the latency of hot scans. We test it on the hot/cold microbenchmark with static placements and migrations disabled, so that only placement matters.

Q2: stalls per miss alone are not enough to measure slowdown, so we expect the AOL-corrected predictor to do better than plain stall cycles and than hotness. We fit it on our own machine and compare the correlations with the measured slowdown.

6 · Questions, Hypotheses and Tests (3/3) — B (∼30 s)

Q3: SOAR, the AOL-based placement, should have a lower slowdown than NoTier and every state-of-the-art system, with the same DRAM. We measure it on the same workloads and the same hardware.

Q4: if Q3 holds, we move to PolyRack. Disaggregated memory is also a fast/slow hierarchy, so we expect SOAR to reduce slowdown there too, after recalibrating the metric for that hardware.

7 · Final Deliverable — B (∼35 s)

Our concrete goal is to answer Q1 to Q3 on our hardware, and Q4 on PolyRack if time permits. The key result will be this figure: on the X-axis, how much of the working set sits in the slow tier; on the Y-axis, the slowdown relative to all-DRAM; one curve per policy. We expect all curves to rise, SOTA sometimes even above NoTier because of useless migrations, and SOAR below both.

Thank you! We have a few open questions for you.

8 · Open questions — B (Q&A, not timed)

Ask the teaching team if the audience doesn't start with their own: server first or PolyRack directly, which workloads, how to access the servers (and whether there is real CXL memory), bare metal or VM.

Likely questions and short answers

- **Why not just measure MLP instead of AOL?** The paper shows MLP alone correlates poorly with slowdown: its benefit shrinks as latency grows. AOL combines both (latency ÷ MLP), and it is cheap to read online from offcore counters.
- **Isn't a hot page still more likely to matter?** Often yes, which is why hotness works in many cases. But hot data is frequently scanned with high MLP, so its slow-tier cost is mostly hidden: frequency alone doesn't tell you the cost.
- **What about ALTO (migration filtering)?** We focus on SOAR, placement, to keep the scope realistic; ALTO is a possible extension if time permits.
- **What do you add beyond the paper?** We test whether its claims hold on different hardware, and we extend it to PolyRack.
- **What if you don't have CXL hardware?** We emulate the slow tier with a remote NUMA node at reduced uncore frequency, as the paper does (90 vs. 190 ns).
- **Will K be the same on your machine?** Not necessarily: a and b depend on the hardware, so we recalibrate them with the sequential and pointer-chasing microbenchmarks. A different shape would itself be a result.
- **What if Q1's hypothesis fails?** It is an anomaly: we sweep the slow-tier latency to see where the effect appears, and check whether SOTA loses because of migration overhead instead.
- **What is your timeline?** Q1 by Oct. 30, Q2 and first SOAR results for the status report (Nov. 12), Q3 by Nov. 27, PolyRack in December if time permits.
- **Why GAPBS / bc-urand?** Large working set with irregular, low-MLP accesses: the paper's stress test for tiered memory.